
Problem Set 3

Both theory and programming questions are due **Thursday, October 6 at 11:59PM**. Please download the .zip archive for this problem set, and refer to the `README.txt` file for instructions on preparing your solutions.

We will provide the solutions to the problem set 10 hours after the problem set is due, which you will use to find any errors in the proof that you submitted. You will need to submit a critique of your solutions by **Friday, October 7th, 11:59PM**. Your grade will be based on both your solutions and your critique of the solutions.

Problem 3-1. [45 points] Range Queries

Microware is preparing to launch a new database product, NoSQL Server, aimed at the real-time analytics market. Web application analytics record information (e.g., the times when users visit the site, or how much does it take the server to produce a HTTP response), and can display the information using a Pretty Graph^{TM1}, so that CTOs can claim that they're using data to back their decisions.

NoSQL Server databases will support a special kind of index, called a *range index*, to speed up the operations needed to build a Pretty GraphTM out of data. Microware has interviewed you during the fall Career Fair, and immediately hired you as a consultant and asked you to help the NoSQL Server team design the range index.

The range index must support fast (sub-linear) insertions, to keep up with Web application traffic. The first step in the Pretty GraphTM algorithm is finding the minimum and maximum values to be plotted, to set up the graph's horizontal axis. So the range index must also be able to compute the minimum and maximum over all keys quickly (in sub-linear time).

- (a) [1 point] Given the constraints above, what data structure covered in 6.006 lectures should be used for the range index? Microware engineers need to implement range indexes, so choose the simplest data structure that meets the requirements.

1. Min-Heap
2. Max-Heap
3. Binary Search Tree (BST)
4. AVL Trees
5. B-Trees

- (b) [1 point] How much time will it take to insert a key in the range index?

1. $O(1)$

¹U.S. patent pending, no. 9,999,999

2. $O(\log(\log N))$
3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$

(c) [1 point] How much time will it take to find the minimum key in the range index?

1. $O(1)$
2. $O(\log(\log N))$
3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$

(d) [1 point] How much time will it take to find the maximum key in the range index?

1. $O(1)$
2. $O(\log(\log N))$
3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$

The main work of the Pretty GraphTM algorithm is drawing the bars in the graph. A bar shows how many data points there are between two values. For example, in order to produce the visitor graph that is the hallmark of Google Analytics, the range index would record each time that someone uses the site, and a bar would count the visiting times between the beginning and the ending of a day. Therefore, the range index needs to support a fast (sub-linear time) $\text{COUNT}(l, h)$ query that returns the number of keys in the index that are between l and h (formally, keys k such that $l \leq k \leq h$).

Your instinct (or 6.006 TA) tells you that $\text{COUNT}(l, h)$ can be easily implemented on top of a simpler query, $\text{RANK}(x)$, which returns the number of keys in the index that are smaller or equal to x (informally, if the keys were listed in ascending order, x 's rank would indicate its position in the sorted array).

(e) [1 point] Assuming $l < h$, and both l and h exist in the index, $\text{COUNT}(l, h)$ is

1. $\text{RANK}(l) - \text{RANK}(h) - 1$
2. $\text{RANK}(l) - \text{RANK}(h)$
3. $\text{RANK}(l) - \text{RANK}(h) + 1$
4. $\text{RANK}(h) - \text{RANK}(l) - 1$
5. $\text{RANK}(h) - \text{RANK}(l)$
6. $\text{RANK}(h) - \text{RANK}(l) + 1$
7. $\text{RANK}(h) + \text{RANK}(l) - 1$
8. $\text{RANK}(h) + \text{RANK}(l)$
9. $\text{RANK}(h) + \text{RANK}(l) + 1$

Problem Set 3

(f) [1 point] Assuming $l < h$, and h exists in the index, but l does not exist in the index, $\text{COUNT}(l, h)$ is

1. $\text{RANK}(l) - \text{RANK}(h) - 1$
2. $\text{RANK}(l) - \text{RANK}(h)$
3. $\text{RANK}(l) - \text{RANK}(h) + 1$
4. $\text{RANK}(h) - \text{RANK}(l) - 1$
5. $\text{RANK}(h) - \text{RANK}(l)$
6. $\text{RANK}(h) - \text{RANK}(l) + 1$
7. $\text{RANK}(h) + \text{RANK}(l) - 1$
8. $\text{RANK}(h) + \text{RANK}(l)$
9. $\text{RANK}(h) + \text{RANK}(l) + 1$

(g) [1 point] Assuming $l < h$, and l exists in the index, but h does not exist in the index, $\text{COUNT}(l, h)$ is

1. $\text{RANK}(l) - \text{RANK}(h) - 1$
2. $\text{RANK}(l) - \text{RANK}(h)$
3. $\text{RANK}(l) - \text{RANK}(h) + 1$
4. $\text{RANK}(h) - \text{RANK}(l) - 1$
5. $\text{RANK}(h) - \text{RANK}(l)$
6. $\text{RANK}(h) - \text{RANK}(l) + 1$
7. $\text{RANK}(h) + \text{RANK}(l) - 1$
8. $\text{RANK}(h) + \text{RANK}(l)$
9. $\text{RANK}(h) + \text{RANK}(l) + 1$

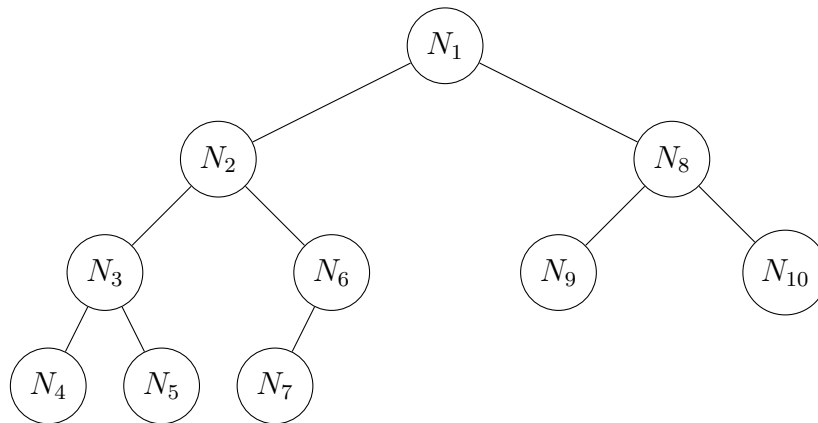
(h) [1 point] Assuming $l < h$, and neither l nor h exist in the index, $\text{COUNT}(l, h)$ is

1. $\text{RANK}(l) - \text{RANK}(h) - 1$
2. $\text{RANK}(l) - \text{RANK}(h)$
3. $\text{RANK}(l) - \text{RANK}(h) + 1$
4. $\text{RANK}(h) - \text{RANK}(l) - 1$
5. $\text{RANK}(h) - \text{RANK}(l)$
6. $\text{RANK}(h) - \text{RANK}(l) + 1$
7. $\text{RANK}(h) + \text{RANK}(l) - 1$
8. $\text{RANK}(h) + \text{RANK}(l)$
9. $\text{RANK}(h) + \text{RANK}(l) + 1$

Now that you know how to reduce a $\text{COUNT}()$ query to a constant number of $\text{RANK}()$ queries, you want to figure out how to implement $\text{RANK}()$ in sub-linear time. None of the tree data structures that you studied in 6.006 supports optimized $\text{RANK}()$ out of the box, but you just remembered that tree data structures can respond to some queries faster if the nodes are cleverly augmented with some information.

- (i) [1 point] In order to respond to $\text{RANK}()$ queries in sub-linear time, each node $node$ in the tree will be augmented with an extra field, $node.\gamma$. Keep in mind that for a good augmentation, the extra information for a node should be computed in $O(1)$ time, based on other properties of the node, and on the extra information stored in the node's subtree. The meaning of $node.\gamma$ is
1. the minimum key in the subtree rooted at $node$
 2. the maximum key in the subtree rooted at $node$
 3. the height of the subtree rooted at $node$
 4. the number of nodes in the subtree rooted at $node$
 5. the rank of $node$
 6. the sum of keys in the subtree rooted at $node$
- (j) [1 point] How many extra *bits* of storage per node does the augmentation above require?
1. $O(1)$
 2. $O(\log(\log N))$
 3. $O(\log N)$
 4. $O(\log^2 N)$
 5. $O(\sqrt{N})$
 6. $O(N)$

The following questions refer to the tree below.



- (k) [1 point] $N_4.\gamma$ is
1. 0
 2. 1
 3. 2
 4. the key at N_4

Problem Set 3

(l) [1 point] $N_3.\gamma$ is

- 1
- 2
- 3
- the key at N_4
- the key at N_5
- the sum of keys at $N_3 \dots N_5$

(m) [1 point] $N_2.\gamma$ is

- 2
- 3
- 4
- 6
- the key at N_4
- the key at N_7
- the sum of keys at $N_3 \dots N_5$

(n) [1 point] $N_1.\gamma$ is

- 3
- 6
- 7
- 10
- the key at N_4
- the key at N_{10}
- the sum of keys at $N_1 \dots N_{10}$

(o) [6 points] Which of the following functions need to be modified to update γ ? If a function does not apply to the tree for the range index, it doesn't need to be modified. (True / False)

- INSERT
- DELETE
- ROTATE-LEFT
- ROTATE-RIGHT
- REBALANCE
- HEAPIFY

(p) [1 point] What is the running time of a COUNT() implementation based on RANK()?

- $O(1)$
- $O(\log(\log N))$

3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$

After the analytics data is plotted using Pretty Graph™, the CEO can hover the mouse cursor over one of the bars, and the graph will show a tooltip with the information represented by that bar. To support this operation, the range index needs to support a $\text{LIST}(l, h)$ operation that returns all the keys between l and h as quickly as possible.

$\text{LIST}(l, h)$ cannot be sub-linear in the worst case, because $\text{LIST}(-\infty, +\infty)$ must return all the keys in the index, which takes $\Omega(n)$ time. However, if LIST only has to return a few elements, we would like it to run in sub-linear time. We formalize this by stating that LIST 's running time should be $T(N) + \Theta(L)$, where L is the length of the list of keys output by LIST , and $T(N)$ is sub-linear.

Inspiration (or your 6.006 TA) strikes again, and you find yourself with the following pseudocode for LIST .

$\text{LIST}(\text{tree}, l, h)$

```

1   $\text{lca} = \text{LCA}(\text{tree}, l, h)$ 
2   $\text{result} = []$ 
3   $\text{NODE-LIST}(\text{lca}, l, h, \text{result})$ 
4  return  $\text{result}$ 
```

$\text{NODE-LIST}(\text{node}, l, h, \text{result})$

```

1  if  $\text{node} == \text{NIL}$ 
2      return
3  if  $l \leq \text{node.key}$  and  $\text{node.key} \leq h$ 
4       $\text{ADD-KEY}(\text{result}, \text{node.key})$ 
5  if  $\text{node.key} \geq l$ 
6       $\text{NODE-LIST}(\text{node.left}, l, h, \text{result})$ 
7  if  $\text{node.key} \leq h$ 
8       $\text{NODE-LIST}(\text{node.right}, l, h, \text{result})$ 
```

$\text{LCA}(\text{tree}, l, h)$

```

1   $\text{node} = \text{tree.root}$ 
2  until  $\text{node} == \text{NIL}$  or  $(l \leq \text{node.key}$  and  $h \geq \text{node.key})$ 
3      if  $l < \text{node.key}$ 
4           $\text{node} = \text{node.left}$ 
5      else
6           $\text{node} = \text{node.right}$ 
7  return  $\text{node}$ 
```

Problem Set 3

(q) [1 point] LCA most likely means

1. last common ancestor
2. lowest common ancestor
3. low cost airline
4. life cycle assessment
5. logic cell array

(r) [1 point] The running time of $\text{LCA}(l, h)$ for the trees used by the range index is

1. $O(1)$
2. $O(\log(\log N))$
3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$

(s) [1 point] Assuming that ADD-KEY runs in $O(1)$ time, and that LIST returns a list of L keys, the running time of the NODE-LIST call at line 3 of LIST is

1. $O(1)$
2. $O(\log(\log N))$
3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$
6. $O(1) + O(L)$
7. $O(\log(\log N)) + O(L)$
8. $O(\log N) + O(L)$
9. $O(\log^2 N) + O(L)$
10. $O(\sqrt{N}) + O(L)$

(t) [1 point] Assuming that ADD-KEY runs in $O(1)$ time, and that LIST returns a list of L keys, the running time of LIST is

1. $O(1)$
2. $O(\log(\log N))$
3. $O(\log N)$
4. $O(\log^2 N)$
5. $O(\sqrt{N})$
6. $O(1) + O(L)$
7. $O(\log(\log N)) + O(L)$
8. $O(\log N) + O(L)$
9. $O(\log^2 N) + O(L)$
10. $O(\sqrt{N}) + O(L)$

- (u) [20 points] Prove that LCA is correct.

Problem 3-2. [55 points] **Digital Circuit Layout**

Your AMDtel internship is off to a great start! The optimized circuit simulator cemented your reputation as an algorithms whiz. Your manager capitalized on your success, and promised to deliver the Bullfield chip a few months ahead of schedule. Thanks to your simulator optimizations, the engineers have finished the logic-level design, and are currently working on laying out the gates on the chip. Unfortunately, the software that verifies the layout is taking too long to run on the preliminary Bullfield layouts, and this is making the engineers slow and unhappy. Your manager is confident in your abilities to speed it up, and promised that you'll "do your magic" again, in "one week, two weeks tops".

A chip consists of logic gates, whose input and output terminals are connected by wires (very thin conductive traces on the silicon substrate). AMDtel's high-yield manufacturing process only allows for horizontal or vertical wires. Wires must not cross each other, so that the circuit will function according to its specification. This constraint is checked by the software tool that you will optimize. The topologies required by complex circuits are accomplished by having dozens of layers of wires that do not touch each other, and the tool works on one layer at a time.

- (a) [1 point] Run the code under the python profiler with the command below, and identify the method that takes up most of the CPU time. If two methods have similar CPU usage times, ignore the simpler one.

```
python -m cProfile -s time circuit2.py < tests/10grid_s.in
```

Warning: the command above can take 15-60 minutes to complete, and bring the CPU usage to 100% on one of your cores. Plan accordingly. If you have installed PyPy successfully, you can replace `python` with `pypy` in the command above for a roughly 2x speed improvement.

What is the name of the method with the highest CPU usage?

- (b) [1 point] How many times is the method called?

The method that has the performance bottleneck is called from the `CrossVerifier` class. Upon reading the class, it seems that the original author was planning to implement a *sweep-line* algorithm, but couldn't figure out the details, and bailed and implemented an inefficient method at the last minute. Fortunately, most of the infrastructure for a fast sweep-line algorithm is still in place. Furthermore, you notice that the source code contains a trace of the working sweep-line algorithm, in the `good_trace.jsonp` file.

Sweep-line algorithms are popular in computational geometry. Conceptually, such an algorithm sweeps a vertical line left to right over the plane containing the input data, and performs operations when the line "hits" point of interest in the input. This is implemented by generating an array containing all the points of interest, and then sorting them according to their position along the horizontal axis (x coordinate).

Problem Set 3

Read the source for `CrossVerifier` to get a feel for how the sweep-line infrastructure is supposed to work, and look at the good trace in the visualizer that we have provided for you. To see the good trace, copy `good_trace.jsonp` to `trace.jsonp`

```
cp good_trace.jsonp trace.jsonp
```

On Windows, use the following command instead.

```
copy good_trace.jsonp trace.jsonp
```

Then use Google Chrome to open `visualizer/bin/visualizer.html`

The questions below refer to the fast sweep-line algorithm shown in `good_trace.jsonp`, not to the slow algorithm hacked together in `circuit2.py`.

- (c) [5 points] The x coordinates of points of interest in the input are (True / False)
1. the x coordinates of the left endpoints of horizontal wires
 2. the x coordinates of the right endpoints of horizontal wires
 3. the x coordinates of midpoints of horizontal wires
 4. the x coordinates where horizontal wires cross vertical wires
 5. the x coordinates of vertical wires
- (d) [1 point] When the sweep line hits the x coordinate of the left endpoint of a horizontal wire
1. the wire is added to the range index
 2. the wire is removed from the range index
 3. a range index query is performed
 4. nothing happens
- (e) [1 point] When the sweep line hits the x coordinate of the right endpoint of a horizontal wire
1. the wire is added to the range index
 2. the wire is removed from the range index
 3. a range index query is performed
 4. nothing happens
- (f) [1 point] When the sweep line hits the x coordinate of the midpoint of a horizontal wire
1. the wire is added to the range index
 2. the wire is removed from the range index
 3. a range index query is performed
 4. nothing happens
- (g) [1 point] When the sweep line hits the x coordinate of a vertical wire
1. the wire is added to the range index

2. the wire is removed from the range index

3. a range index query is performed

4. nothing happens

(h) [1 point] What is a good invariant for the sweep-line algorithm?

1. the range index holds all the horizontal wires to the left of the sweep line

2. the range index holds all the horizontal wires “stabbed” by the sweep line

3. the range index holds all the horizontal wires to the right of the sweep line

4. the range index holds all the wires to the left of the sweep line

5. the range index holds all the wires to the right of the sweep line

(i) [1 point] When a wire is added to the range index, what is its corresponding key?

1. the x coordinate of the wire’s midpoint

2. the y coordinate of the wire’s midpoint

3. the segment’s length

4. the x coordinate of the point of interest that will remove the wire from the index

Modify `CrossVerifier` in `circuit2.py` to implement the sweep-line algorithm discussed above. If you maintain the current code structure, you’ll be able to use our visualizer to debug your implementation. To use our visualizer, first produce a trace.

```
TRACE=jsonp python circuit2.py < tests/5logo.in > trace.jsonp
```

On Windows, run the following command instead.

```
circuit2_jsonp.bat < tests/5logo.in > trace.jsonp
```

Then use Google Chrome to open `visualizer/bin/visualizer.html`

(j) [1 point] Run your modified code under the python profiler again, using the same test case as before, and identify the method that takes up the most CPU time.

What is the name of the method with the highest CPU usage? If two methods have similar CPU usage times, ignore the simpler one.

(k) [1 point] How many times is the method called?

(l) [40 points] Modify `circuit2.py` to implement a data structure that has better asymptotic running time for the operation above. Keep in mind that the tool has two usage scenarios:

- Every time an engineer submits a change to one of the Bullhorn wire layers, the tool must analyze the layer and report the number of wire crossings. In this late stage of the project, the version control system will automatically reject the engineer’s change if it causes the number of wire crossings to go up over the previous version.

Problem Set 3

- Engineers working on the wiring want to see the pairs of wires that intersect, so they know where to focus their efforts. To activate this detailed output, run the tool using the following command.

```
TRACE=list python circuit2.py < tests/6list_logo.in
```

On Windows, run the following command instead.

```
circuit2_list.bat < tests/6list_logo.in
```

When your code passes all tests, and runs reasonably fast (the tests should complete in less than 60 seconds on any reasonably recent computer), upload your modified `circuit.py` to the course submission site.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.006 Introduction to Algorithms
Fall 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.